

Example 1.66: Suppose that we received two binary numbers in Parity Code. The 1st binary number is coded in odd parity and the received data is " $1001_{(\text{number})} 0_{(\text{check bit})}$ ". The 2nd binary number is coded in even parity and the received data is " $0110_{(\text{number})} 1_{(\text{check bit})}$ ". Please justify the correctness of the received data.

Solution:

(1) The total number of "1s" in the received data " $1001_{(\text{number})} 0_{(\text{check bit})}$ " is "2", but "2" is even. The received data conflicts with the odd parity. Thus, the 1st binary number is corrupted in the data transmission.

(2) The total number of "1s" in the received data " $0110_{(\text{number})} 1_{(\text{check bit})}$ " is "3", but "3" is odd. The received data conflicts with the even parity. Thus, the 2nd binary number is corrupted in the data transmission.

Example 1.67: Suppose that we send the binary number " $(1010)_2$ " and it is coded in odd parity. During the data transmission, the data bits are corrupted twice. First, the most significant bit of " $(1010)_2$ " is changed from "1" into "0". Second, the least significant bit of " $(1010)_2$ " is changed from "0" into "1". Please justify the correctness of the received data.

Solution:

(1) The original binary number coded in odd parity is " $1010_{(\text{number})} 1_{(\text{check bit})}$ ".

(2) When the most significant bit of the binary number is corrupted, the data becomes " $0010_{(\text{number})} 1_{(\text{check bit})}$ ".

(3) When the least significant bit of the binary number is corrupted, the data becomes " $0011_{(\text{number})} 1_{(\text{check bit})}$ ".

(4) The total number of "1s" in the received data " $0011_{(\text{number})} 1_{(\text{check bit})}$ " is "3", and "3" is odd. In this case, the received data obeys the odd parity. But the received data is corrupted.

From Example 1.67, we find the drawbacks of the Parity Code: (a) If any even number of bits are corrupted in the data transmission, the Parity Code is unable to detect the error. (b) The Parity Code is unable to locate the error bit from the received data.

Exercise

- 1.1 Convert the following binary numbers to the equivalent octal numbers.
 - (a) $(11011000)_2$
 - (b) $(101110.0111)_2$
- 1.2 Convert the following binary numbers to the equivalent hexadecimal numbers.
 - (a) $(1101010)_2$
 - (b) $(10110110)_2$
 - (c) $(10101.1101)_2$
- 1.3 Convert the following octal numbers to the equivalent binary numbers.
 - (a) $(174002)_8$
 - (b) $(67.23)_8$

- 1.4 Convert the following hexadecimal numbers to the equivalent binary numbers.

- (a) $(F3A4)_{16}$
- (b) $(AB2D)_{16}$
- (c) $(25C.38)_{16}$

- 1.5 Convert the following hexadecimal numbers to the equivalent octal numbers.

- (a) $(1024)_{16}$
- (b) $(6E6A)_{16}$
- (c) $(ABCE)_{16}$
- (d) $(C351)_{16}$
- (e) $(8E36.7A)_{16}$
- (f) $(CEAD.BEEF)_{16}$

- 1.6 Convert the following binary numbers to the equivalent decimal numbers.

- (a) $(1101010)_2$
- (b) $(10110110)_2$
- (c) $(10101.1101)_2$

- 1.7 Convert the following octal numbers to the equivalent decimal numbers.

- (a) $(7155)_8$
- (b) $(174002)_8$
- (c) $(66.24)_8$

- 1.8 Convert the following hexadecimal numbers to the equivalent decimal numbers.

- (a) $(F3A4)_{16}$
- (b) $(AB3C)_{16}$
- (c) $(15D.38)_{16}$

- 1.9 Convert the following decimal numbers to the equivalent binary numbers.

- (a) $(124)_{10}$
- (b) $(208)_{10}$
- (c) $(133)_{10}$

- 1.10 Convert the following decimal numbers to the equivalent octal numbers.

- (a) $(1434)_{10}$
- (b) $(3488)_{10}$
- (c) $(9713)_{10}$

- 1.11 Convert the following decimal numbers to the equivalent hexadecimal numbers.

- (a) $(23850)_{10}$
- (b) $(57191)_{10}$
- (c) $(65112)_{10}$

- 1.12 Write the signed-magnitude, the one's complement, and the two's complement of the following binary numbers.

- (a) $(+1011)_2$
- (b) $(-1011)_2$
- (c) $(+0.0011)_2$
- (d) $(-0.0011)_2$

- 1.13 Calculate the addition of the following binary numbers in the two's complement.

- (a) $X_1 = (-0110110)_2$
- (b) $X_2 = (-0011110)_2$

F	CD				
		00	01	11	10
AB	00	1	1	0	1
	01	1	1	0	1
	11	0	0	0	0
	10	1	1	0	1

Figure 2.54 K-map of Function "F(A, B, C, D)" of Example 2.45

(2) We draw the K-circles on "0" blocks of the K-map.

First, we draw the K-circle on "0" blocks in the 3rd row of the K-map in Figure 2.55. The variable "C" covers "0" and "1", so "C" is offset. The variable "D" covers "0" and "1", so "D" is offset. The variable "A" is "1" and "B" is "1", so we write the sum term " $\bar{A} + \bar{B}$ " for the K-circle.

Second, we draw the K-circle on "0" blocks in the 3rd column of the K-map in Figure 2.55. The variable "A" covers "0" and "1", so "A" is offset. The variable "B" covers "0" and "1", so "B" is offset. The variable "C" is "1" and "D" is "1", so we write the sum term " $\bar{C} + \bar{D}$ " for the K-circle.

Third, we write the product of the sum terms " $\bar{A} + \bar{B}$ " and " $\bar{C} + \bar{D}$ ". Therefore, we obtain the simplest "OR-AND" function of the K-map.

$$F(A, B, C, D) = (\bar{A} + \bar{B}) \cdot (\bar{C} + \bar{D})$$

F	CD				
		00	01	11	10
AB	00	1	1	0	1
	01	1	1	0	1
	11	0	0	0	0
	10	1	1	0	1

Figure 2.55 K-circles of Function "F(A, B, C, D)" of Example 2.45

Exercise

2.1 Write the truth table of the following logic functions.

(a) $F = \bar{X}Y + \bar{X}Z + XYZ$

(b) $F = A\bar{B} + \bar{B}C + C\bar{D} + C\bar{A} + ABCD$

(c) $F = (\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + D) \cdot (B + C + \bar{D}) \cdot (A + B + C + D) \cdot (\bar{A} + \bar{B} + \bar{C} + \bar{D})$

2.2 Write the canonical "AND-OR" functions (i. e., sum of minterms) and the canonical "OR-AND" (i. e., product of maxterms) functions of the following logic functions.

(a) $F = \bar{X} + YZ + XY\bar{Z}$

(b) $F = (V + W\bar{X}) \cdot (V + W + X)$

2.3 Write the simplest "AND-OR" functions of the following logic functions by K-map.

(a) $F(X, Y, Z) = \sum m(1, 2, 3, 5, 6, 7)$

(b) $F(W, X, Y) = \prod M(1, 5, 6, 7)$

2.4 Write the simplest "AND-OR" functions of the following logic functions by K-map.

(a) $F(W, X, Y, Z) = \sum m(1, 4, 5, 6, 7, 8, 9, 14, 15)$

(b) $F(W, X, Y, Z) = \sum m(0, 1, 6, 7, 8, 9, 11, 14, 15)$

(c) $F(A, B, C, D) = \prod M(4, 5, 12, 13, 15)$

(d) $F(A, B, C, D) = \sum m(4, 5, 6, 7, 11, 13, 14, 15)$

3.2 Basic Logic Gates Implem

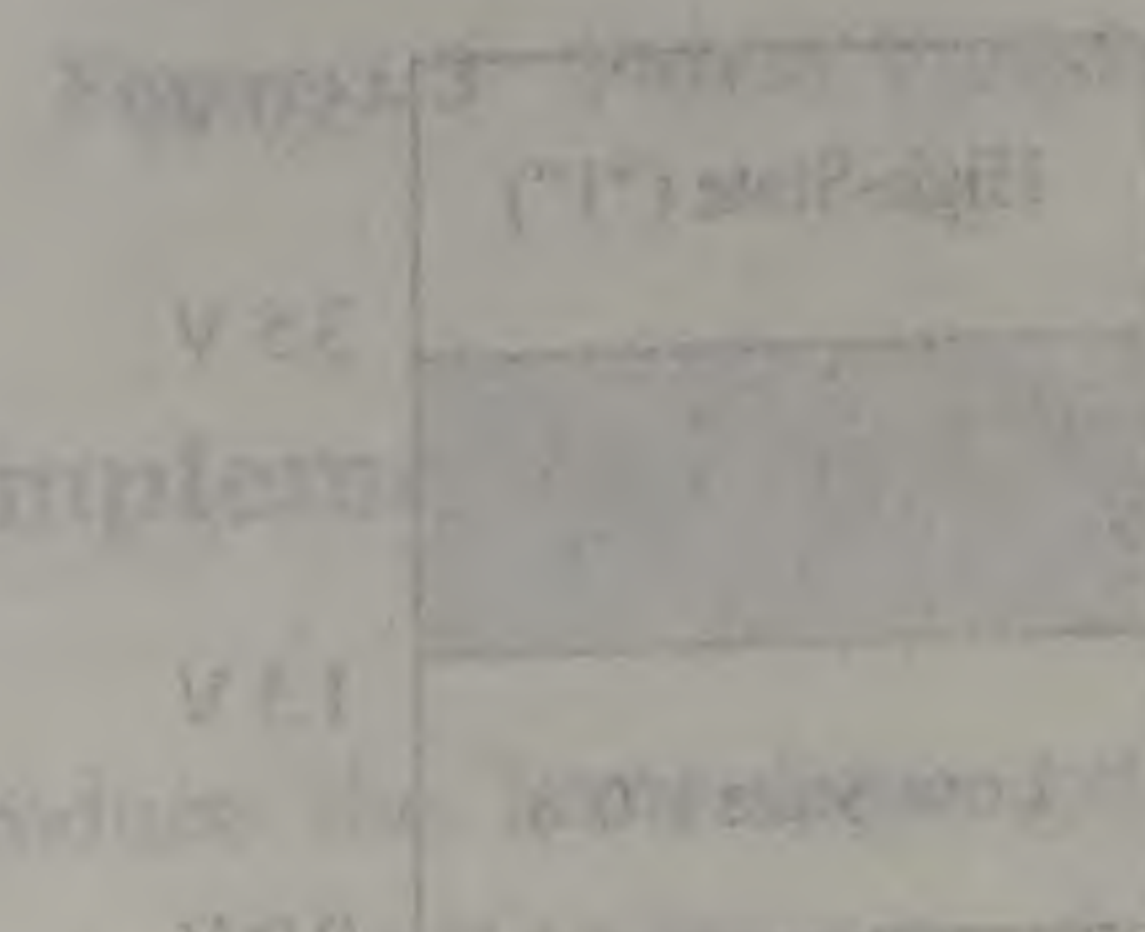


Figure 3.1 Logic Level of CMOS Transistor Working in the "0" State

According to the material type of semiconductor, the CMOS transistor is divided into two categories: PMOS (P-channel MOS) transistor and the NMOS (N-channel MOS) transistor. PMOS transistor has three terminals: "gate", "source", and "drain". The "gate" terminal is connected to the input signal, the "source" terminal is connected to the "VDD" terminal, and the "drain" terminal is connected to the output node. The NMOS transistor is connected to the "gate" terminal, the "source" terminal is connected to the "ground" terminal, and the "drain" terminal is connected to the output node. The output node is also connected to a load capacitor. The output voltage is shown as a square wave between 0V and 2.5V.

(2) The residual driving capacity of CMOS 74HCT is calculated as:

$$\begin{aligned} R_c &= I_{OHmax} - F_o \times I_{IHmax} \\ &= 4.0 \text{ mA} - 10 \times 0.02 \text{ mA} \\ &= 4.0 \text{ mA} - 0.2 \text{ mA} \\ &= 3.8 \text{ mA} \end{aligned}$$

Therefore, the residual driving capacity of CMOS 74HCT is "3.8 mA".

Exercise

3.1 The voltage level of TTL 74LS00 is summarized in Table 3.11. Please calculate the DC noise margin:

- (1) When the output voltage of TTL 74LS00 is high.
- (2) When the output voltage of TTL 74LS00 is low.

Table 3.11 Voltage Level of TTL 74LS00

Logic Family	Voltage Level			
	V_{OHmin}	V_{IHmin}	V_{ILmax}	V_{OLmax}
TTL 74LS00	2.7	2.0	0.8	0.5

3.2 The voltage levels of CMOS 74HCT and TTL 74ALS are summarized in Table 3.12. Suppose that TTL 74ALS drives CMOS 74HCT. Please calculate the DC noise margin:

- (1) When the output voltage of TTL 74ALS is high.
- (2) When the output voltage of TTL 74ALS is low.

Table 3.12 Voltage Levels of CMOS 74HCT and TTL 74ALS

Logic Family	Voltage Level			
	V_{OHmin}	V_{IHmin}	V_{ILmax}	V_{OLmax}
CMOS 74HCT	3.84	2.0	0.8	0.33
TTL 74ALS	2.70	2.0	0.8	0.50

3.3 The current levels of CMOS 74HCT and TTL 74AS are summarized in Table 3.13. Suppose that CMOS 74HCT drives TTL 74AS. Please calculate:

- (1) The overall fanout of CMOS 74HCT.
- (2) The residual driving capacity of CMOS 74HCT.

Table 3.13 Current Levels of CMOS 74HCT and TTL 74AS

Logic Family	Current Level			
	I_{OHmax}	I_{OLmax}	I_{IHmax}	I_{ILmax}
CMOS 74HCT	4 mA	4 mA	1.00 μ A	1.0 μ A
TTL 74AS	2 mA	20 mA	0.02 mA	0.5 mA

3.4 Analyze the CMOS digital circuit in Figure 3.28:

- (1) Write the voltage table and the logic function.
- (2) Draw the logic circuit diagram of the logic function.

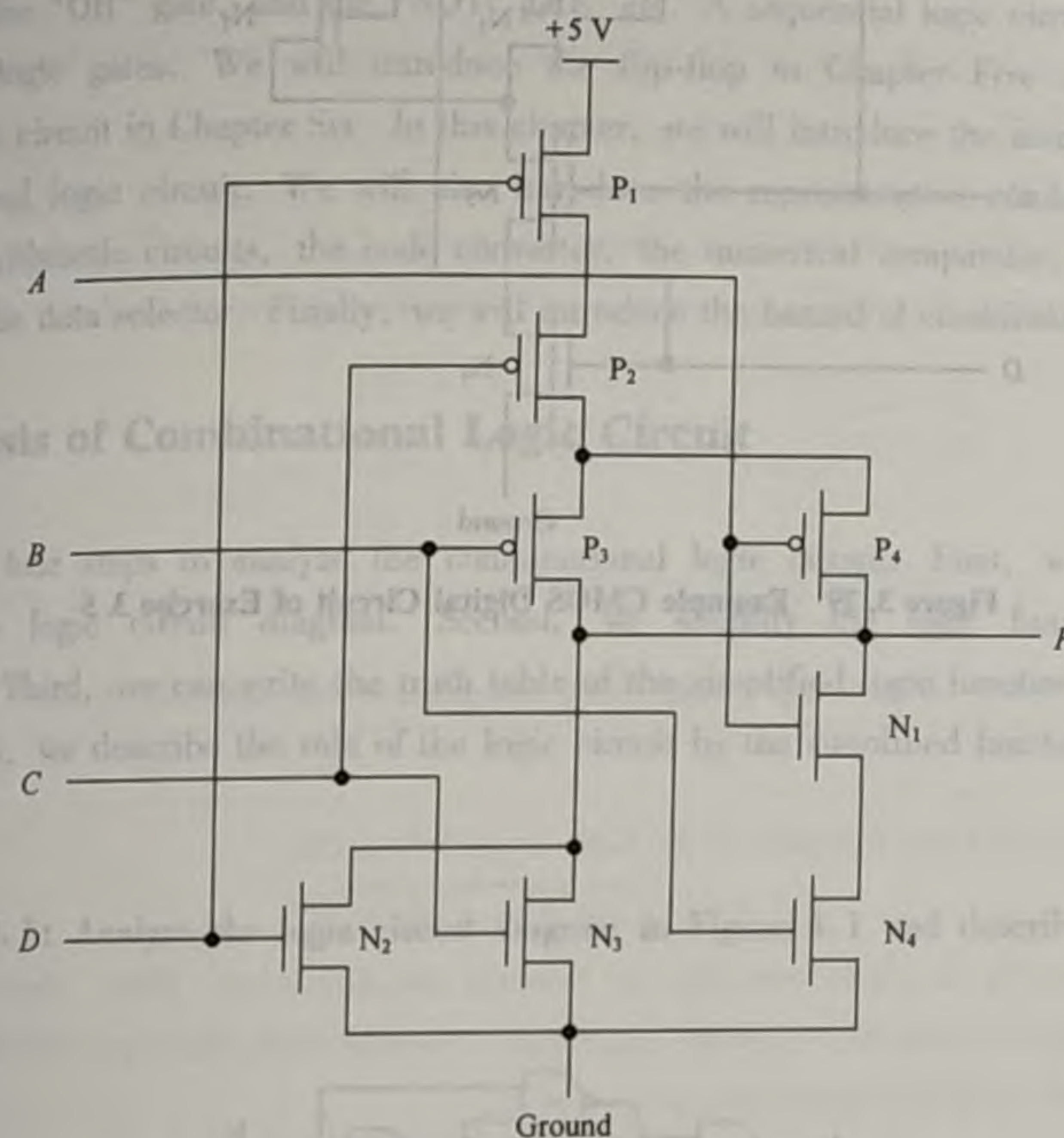


Figure 3.28 Example CMOS Digital Circuit of Exercise 3.4

3.5 Analyze the CMOS digital circuit in Figure 3.29:

- (1) Write the voltage table and the logic function.
- (2) Draw the logic circuit diagram of the logic function.

$$\begin{aligned}
 F &= A \cdot B + \bar{A} \cdot C + BC \\
 &= A \cdot 1 + \bar{A} \cdot 1 + 1 \cdot 1 \\
 &= A + \bar{A} + 1 \\
 &= 1
 \end{aligned}$$

If the input variable "B" is the binary number "1", and the input variable "C" is the binary number "1", the output variable "F" of the logic function is the binary number "1". That is to say, the redundant product term " $BC = 1$ " eliminates the race of both input variables "A" and " $\bar{A}$ ". Therefore, the hazard of the logic function "F" is eliminated.

Exercise

4.1 Write the simplest "AND-OR" functions of the following logic functions by K-map.

(a) $F(W, X, Y, Z) = \sum m(0, 1, 3, 5, 14) + d(15)$

(b) $F(W, X, Y, Z) = \sum m(0, 1, 2, 8, 11) + d(3, 9)$

(c) $F(A, B, C, D) = \sum m(4, 6, 7, 9, 13) + d(5)$

(d) $F(A, B, C, D) = \sum m(1, 5, 12, 13, 14, 15) + d(9)$

(e) $F(W, X, Y, Z) = \sum m(4, 5, 9, 13, 15) + d(0, 1, 11, 12)$

4.2 Detect and eliminate the hazard of the following logic functions.

(a) $F = WX + \bar{W} \cdot \bar{Y} + \bar{X}Y$

(b) $F = W\bar{X} \cdot \bar{Y} + X\bar{Y}Z$

4.3 Use the 3-8 decoder and the "NAND" gate to implement the following logic functions.

(a) $F(X, Y, Z) = \sum m(2, 4, 6, 7)$

(b) $F(A, B, C) = \prod M(4, 5, 6, 7)$

4.4 Use the 4-to-1 line data selector to implement the following logic functions.

(a) $F(A, B, C, D) = \sum m(0, 2, 4, 10, 12)$

(b) $F(W, X, Y, Z) = \sum m(2, 3, 4, 5, 10, 12, 14)$



the characteristic equation " Q^{n+1} " of R-S flip-flop, which is illustrated as follows. In order to use fewer types of logic gates, we change the "OR" operation to "NAND" operation by De Morgan's law. Therefore, in the excitation equation " D ", we obtain two "NAND" operations.

D Flip-flop: $Q^{n+1} = D$

R-S Flip-flop: $Q^{n+1} = S + \bar{R}Q^n$

Excitation Equation of D Flip-flop: $D = S + \bar{R}Q^n = \overline{\overline{S} \cdot \overline{R} \cdot \overline{Q^n}}$

The logic circuit diagram of the R-S flip-flop implemented by the D flip-flop is illustrated in Figure 5.44.

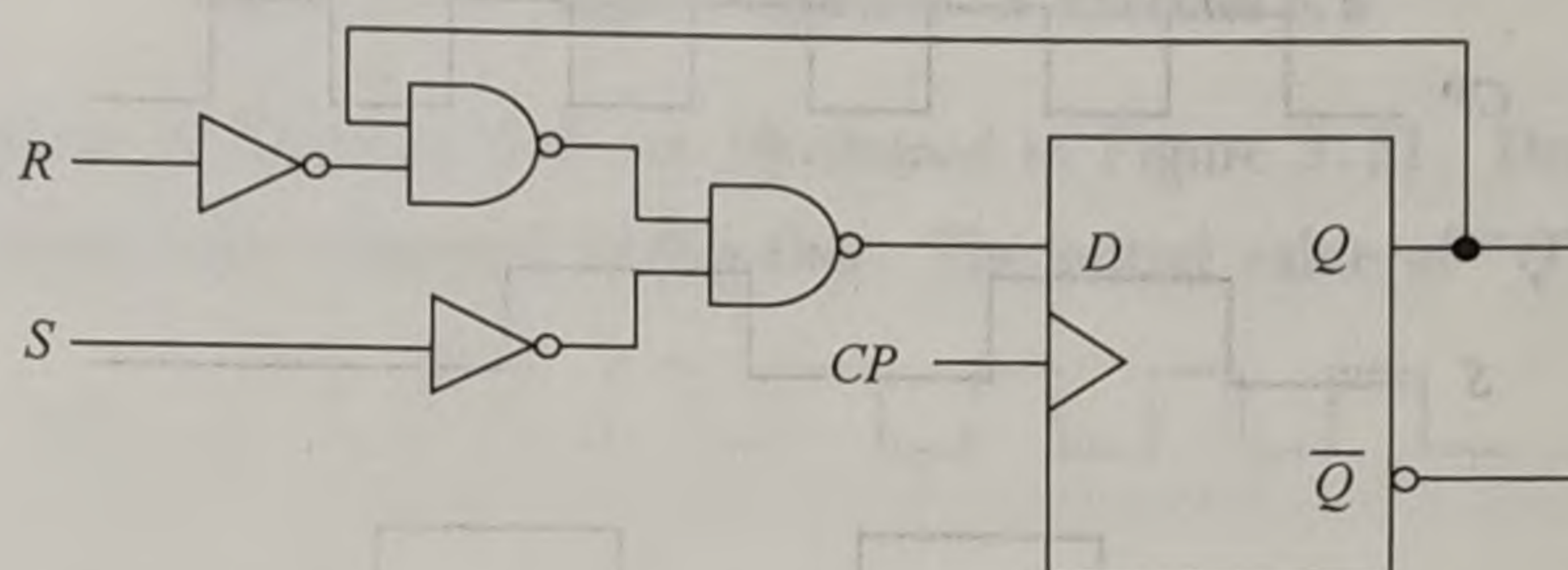


Figure 5.44 R-S Flip-flop Implemented by the D Flip-flop

5.7.5 Conversion from D Flip-Flop to T Flip-flop

We can implement the T flip-flop by the D flip-flop. We set the excitation equation " D " to the characteristic equation " Q^{n+1} " of T flip-flop, which is illustrated as follows.

D Flip-flop: $Q^{n+1} = D$

T Flip-flop: $Q^{n+1} = T \oplus Q^n$

Excitation Equation of D Flip-flop: $D = T \oplus Q^n$

The logic circuit diagram of the T flip-flop implemented by the D flip-flop is illustrated in Figure 5.45.

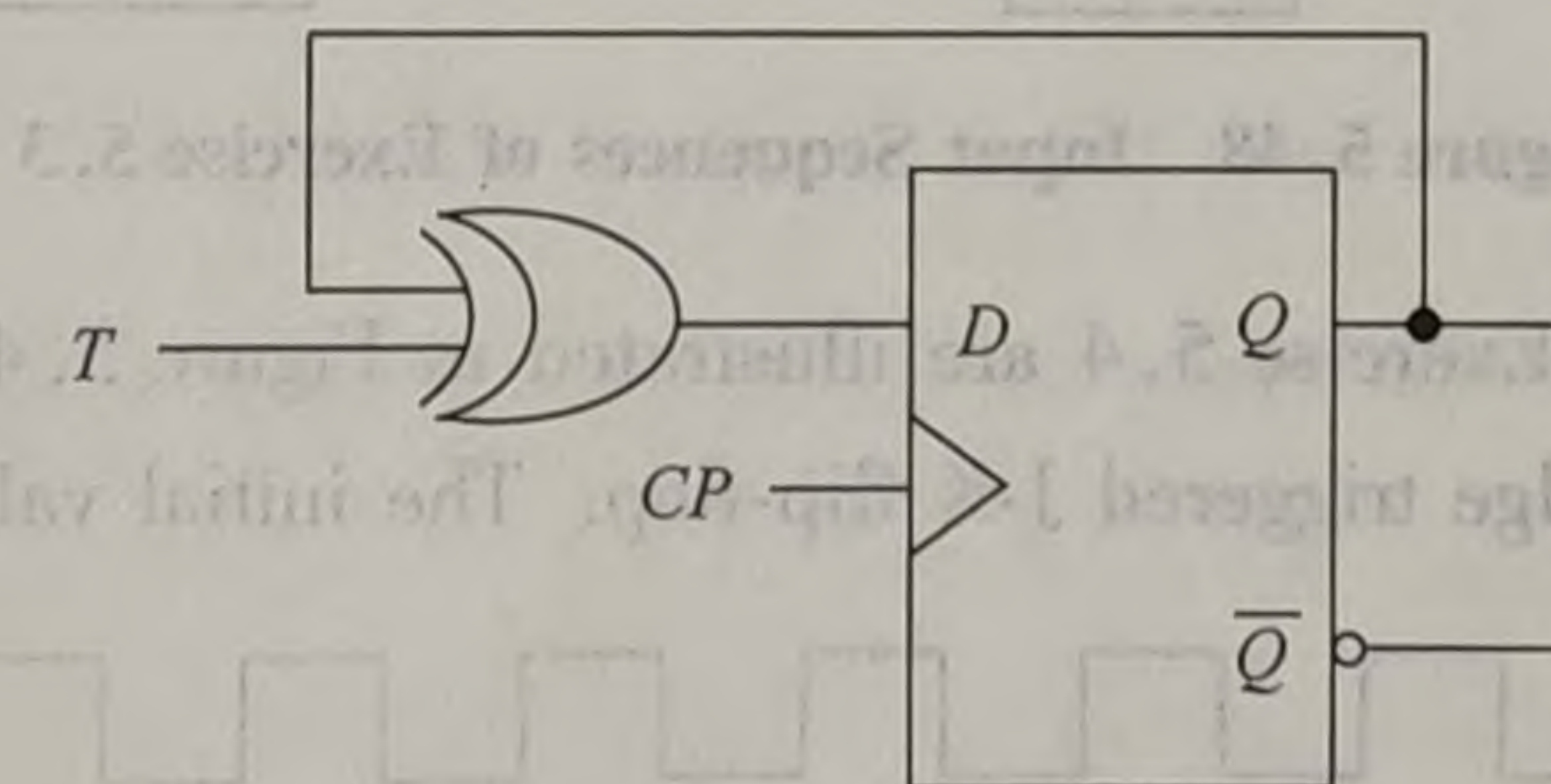


Figure 5.45 T Flip-flop Implemented by the D Flip-flop

Exercise

- 5.1 The input sequences of Exercise 5.1 are illustrated in Figure 5.46. Draw the time sequence diagram of the R-S latch composed of "NAND" gates. The initial value of " Q " is "0".

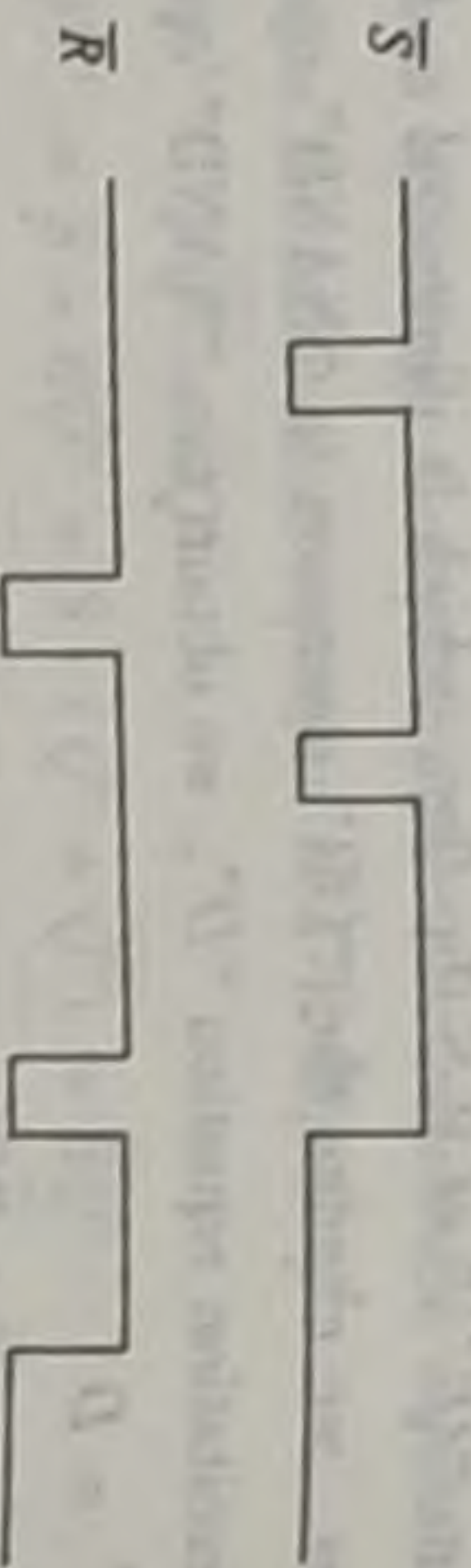


Figure 5.46 Input Sequences of Exercise 5.1

5.2 The input sequences of Exercise 5.2 are illustrated in Figure 5.47. Draw the time sequence diagram of the gated R-S latch. The initial value of "Q" is "0".



Figure 5.47 Input Sequences of Exercise 5.2

5.3 The input sequences of Exercise 5.3 are illustrated in Figure 5.48. Draw the time sequence diagram of the master/slave J-K flip-flop. The initial value of "Q" is "0".

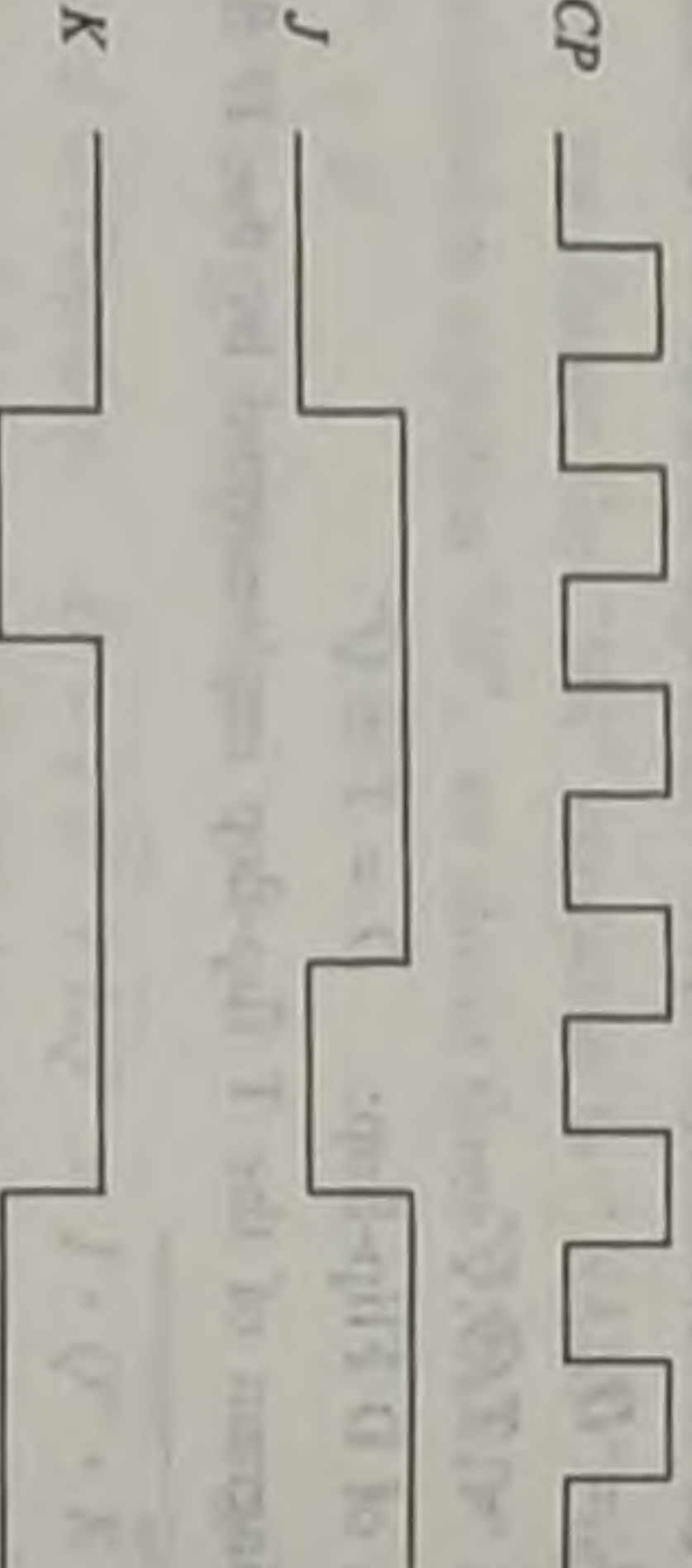


Figure 5.48 Input Sequences of Exercise 5.3

5.4 The input sequences of Exercise 5.4 are illustrated in Figure 5.49. Draw the time sequence diagram of the falling-edge triggered J-K flip-flop. The initial value of "Q" is "0".



Figure 5.49 Input Sequences of Exercise 5.4

5.5 The input sequences of Exercise 5.5 are illustrated in Figure 5.50. Draw the time sequence diagram of the falling-edge triggered J-K flip-flop. The initial value of "Q" is "0".

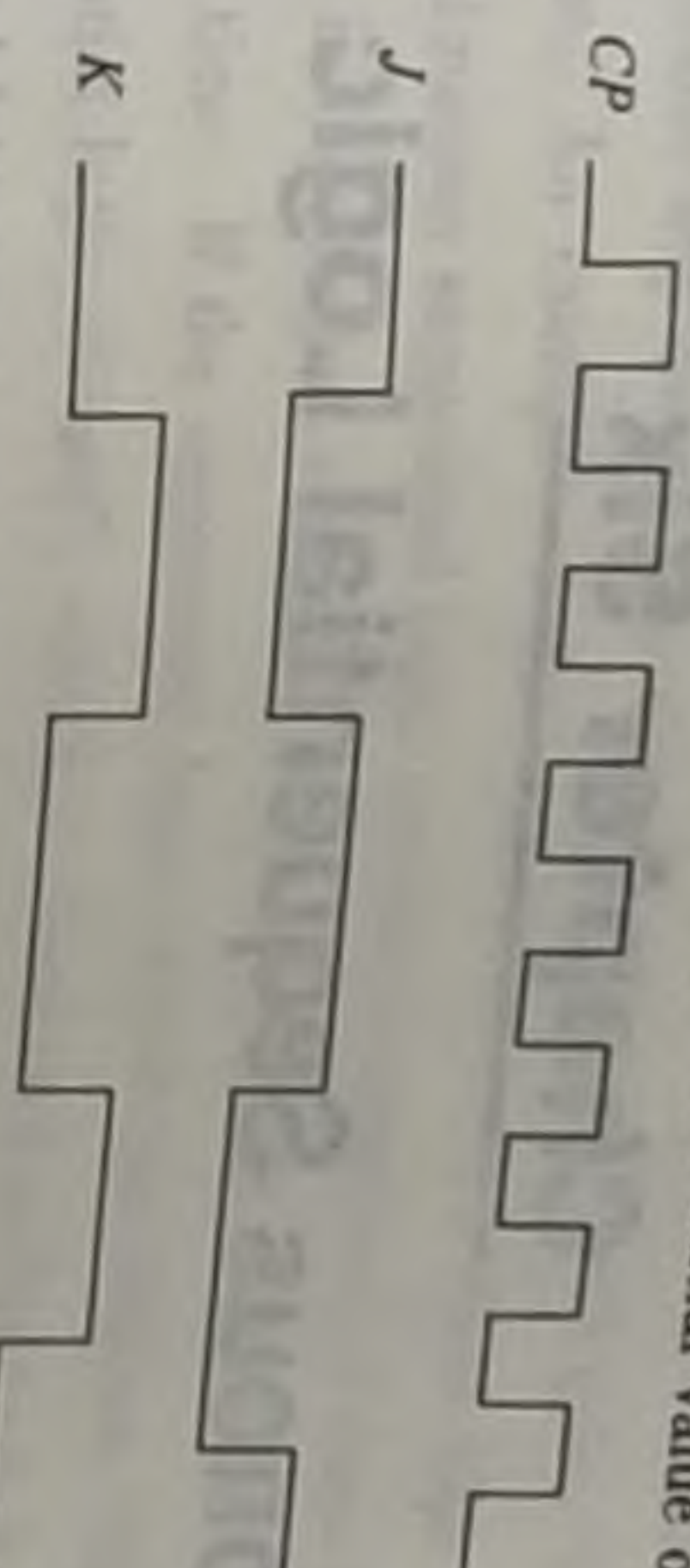


Figure 5.50 Input Sequences of Exercise 5.5

5.6 The input sequences of Exercise 5.6 are illustrated in Figure 5.51. Draw the time sequence diagram of the rising-edge triggered D flip-flop. The initial value of "Q" is "0".

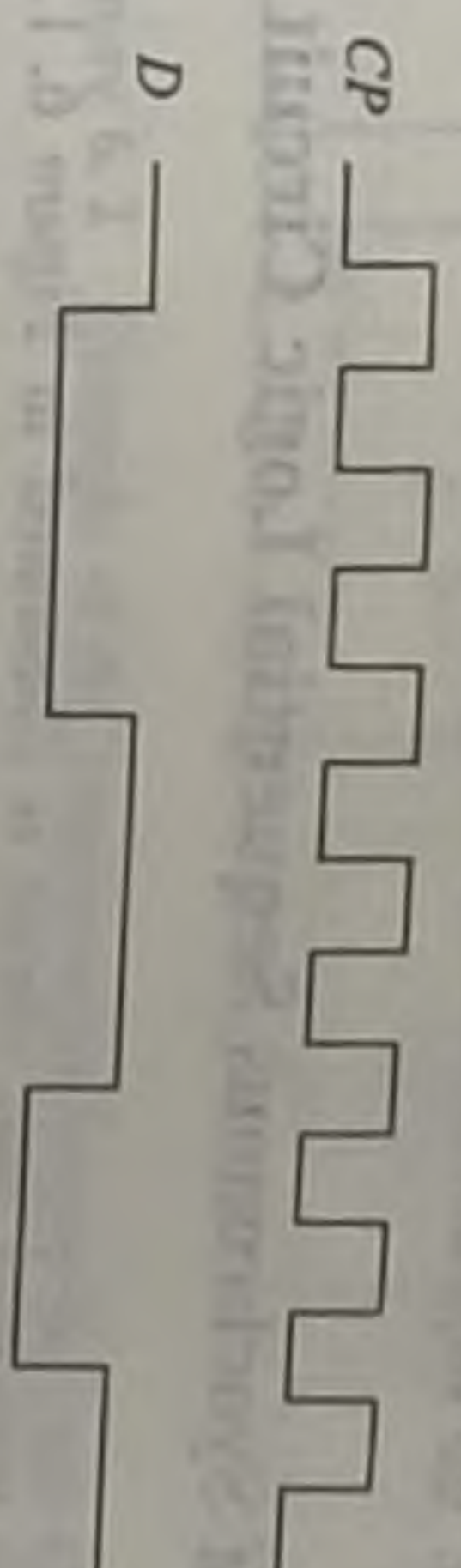


Figure 5.51 Input Sequences of Exercise 5.6

5.7 The input sequences of Exercise 5.7 are illustrated in Figure 5.52. Draw the time sequence diagram of the gated D latch. The initial value of "Q" is "0".



Figure 5.52 Input Sequences of Exercise 5.7

Exercise

- 6.1 Analyze the synchronous sequential logic circuit in Figure 6.91. (a) Write the output equation, the excitation equation and the state equation. (b) Write the state transition truth table and draw the state transition diagram. (c) Describe the role of circuit.

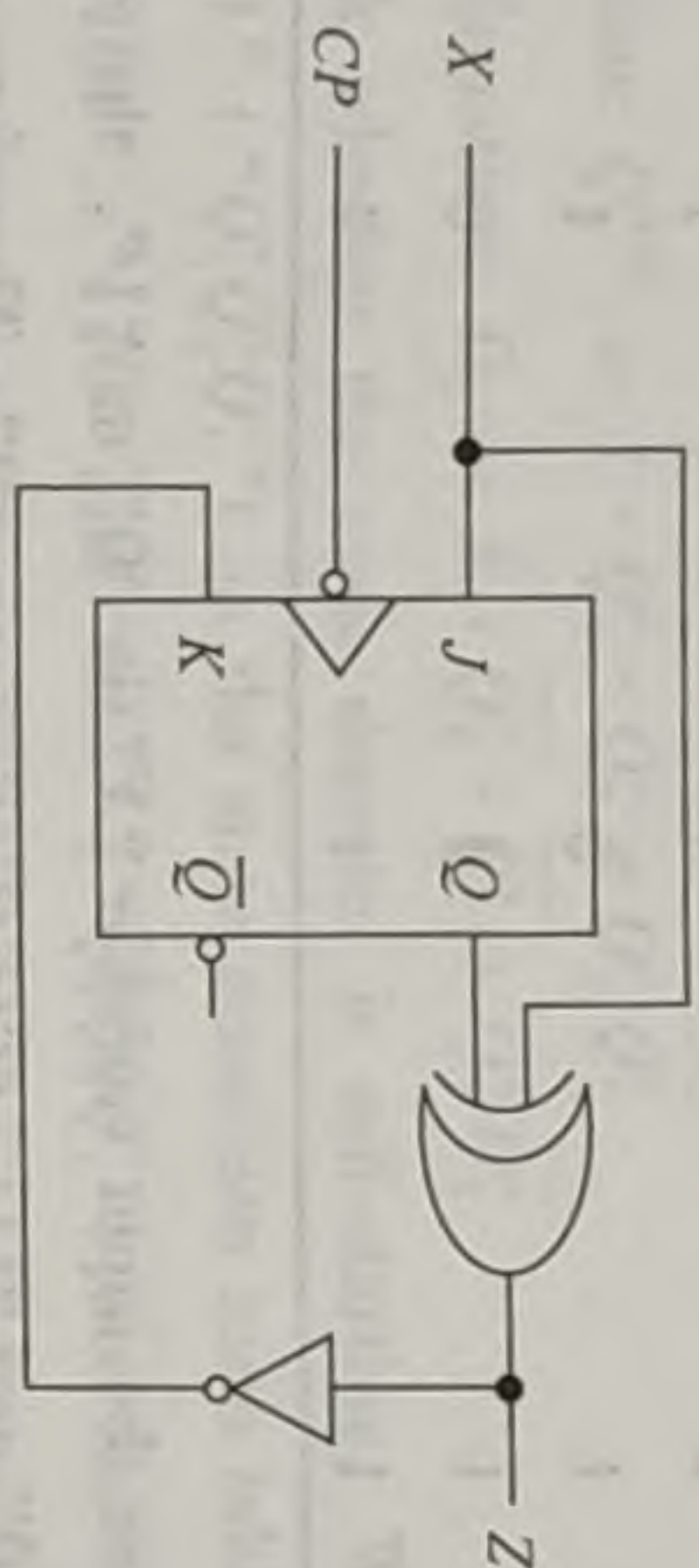


Figure 6.91 Synchronous Sequential Logic Circuit of Exercise 6.1

- 6.2 Analyze the synchronous sequential logic circuit in Figure 6.92. (a) Write the excitation equation and the state equation. (b) Write the state transition truth table and draw the state transition diagram. (c) Describe the role of circuit.

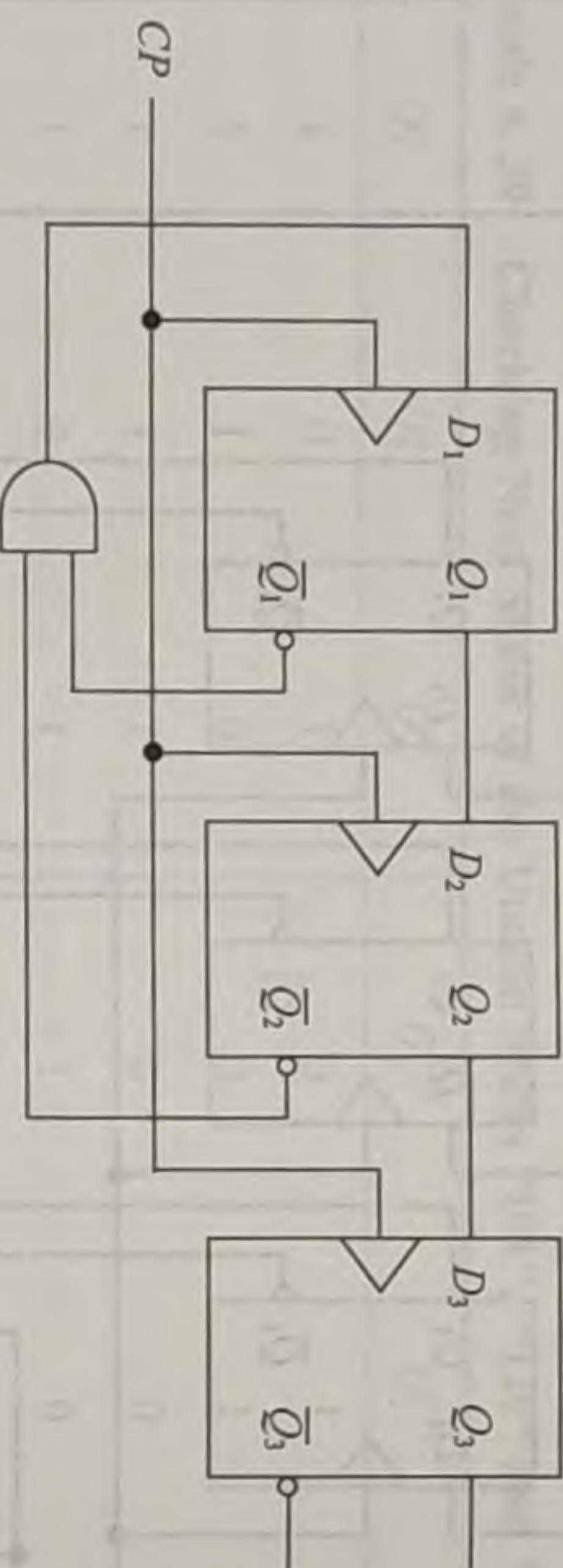


Figure 6.92 Synchronous Sequential Logic Circuit of Exercise 6.2

- 6.3 Analyze the synchronous sequential logic circuit in Figure 6.93. (a) Write the excitation equation and the state equation. (b) Write the state transition truth table and draw the state transition diagram. (c) Describe the role of circuit.

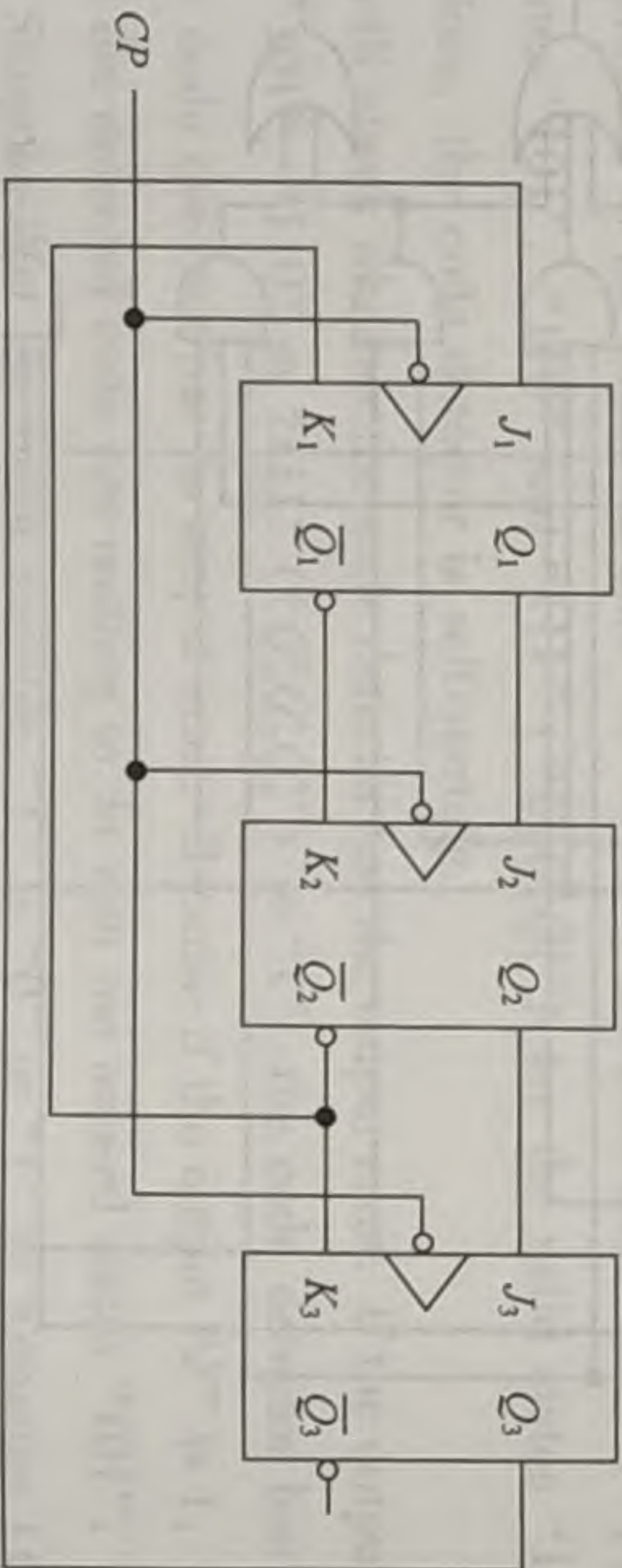


Figure 6.93 Synchronous Sequential Logic Circuit of Exercise 6.3

- 6.4 Analyze the synchronous sequential logic circuit in Figure 6.94. (a) Write the excitation equation and the state equation. (b) Write the state transition truth table and draw the state transition diagram. (c) Describe the role of circuit.

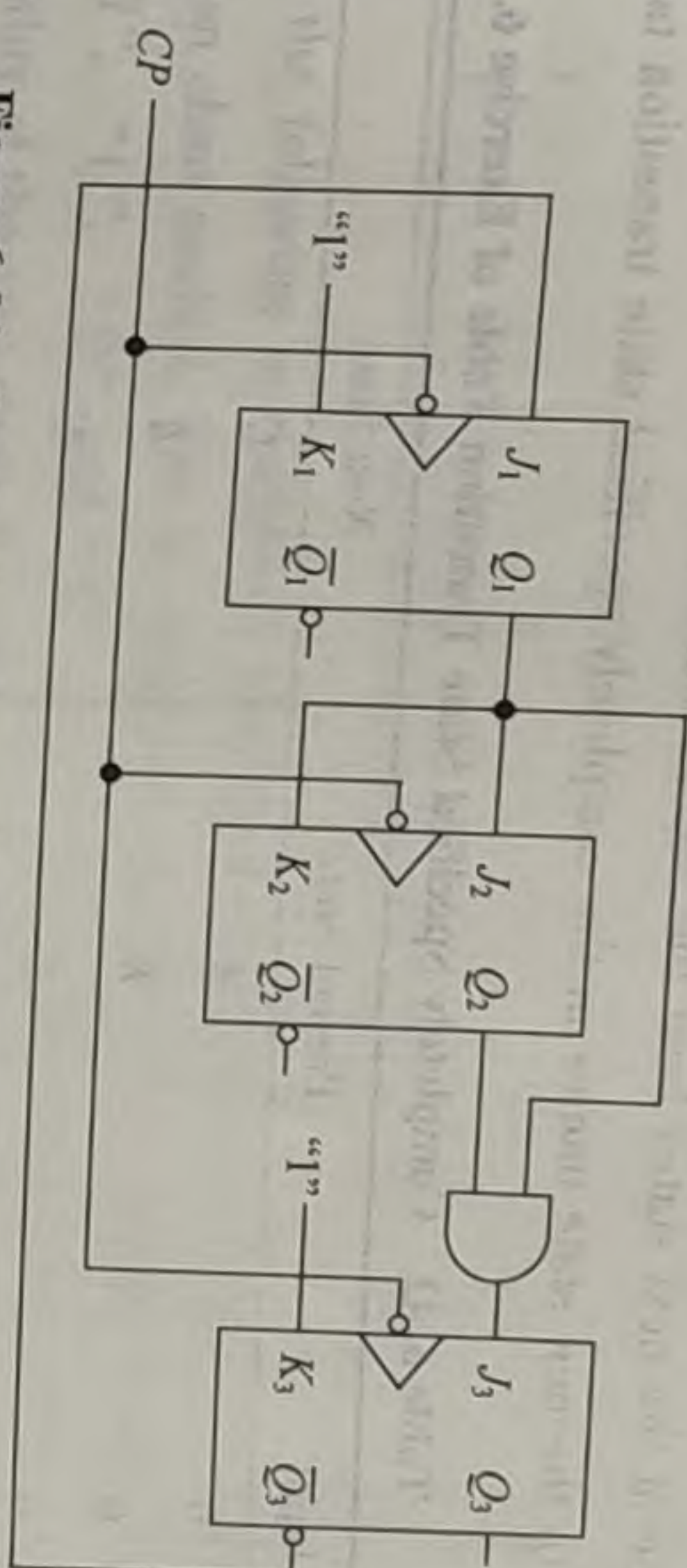


Figure 6.94 Synchronous Sequential Logic Circuit of Exercise 6.4

- 6.5 Design the sequence detector by J-K flip-flop. It detects the sequence of binary numbers "1011", which can be overlapped. (a) Draw the state transition diagram and write the state transition truth table. (b) Draw the K-map and the K-circles. (c) Write the state equation, the excitation equation, and the output equation. (d) Draw the logic circuit diagram.

- 6.6 Simplify the following completely specified state transition table in Table 6.41 by the implication chart method. The completely specified state transition table contains 7 states: "A", "B", "C", "D", "E", "F" and "G". (a) Draw the implication chart. (b) Check the equivalence of the next states from the implication chart. (c) Replace the equivalent state classes with the new state names in the completely specified state transition table.

Table 6.41 Completely Specified State Transition Table of Exercise 6.6

Input	Present State	Next State	Output
0	A	A	0
0	B	B	0
0	C	D	1
0	D	G	1
0	E	E	0
0	F	F	0
0	G	C	0
1	A	G	1
1	B	D	0
1	C	E	0
1	D	E	1
1	E	G	1
1	F	D	1
1	G	F	0

6.7 Simplify the following completely specified state transition table in Table 6.42 by the implication chart method. The completely specified state transition table contains 6 states: "A", "B", "C", "D", "E" and "F". (a) Draw the implication chart. (b) Check the equivalence of the next states from the implication chart. (c) Replace the equivalent state classes with the new state names in the completely specified state transition table.

Table 6.42 Completely Specified State Transition Table of Exercise 6.7

Input	Present State	Next State	Output
0	A	C	1
0	B	B	0
0	C	C	1
0	D	D	0
0	E	E	0
0	F	F	0
1	A	D	1
1	B	C	1
1	C	A	0
1	D	C	0
1	E	C	0
1	F	C	0

6.8 Simplify the following in-completely specified state transition table in Table 6.43 by the implication chart method. The in-completely specified state transition table contains 6 states: "A", "B", "C", "D", "E" and "F". (a) Draw the implication chart. (b) Check the compatibility of the next states from the implication chart. (c) Draw the merger diagram of the compatible state pairs. (d) Check the coverage property of the compatible state classes. (e) Check the closure property of the compatible state classes. (f) Replace the compatible state classes with the new state names in the in-completely specified state transition table.

Table 6.43 In-completely Specified State Transition Table of Exercise 6.8

Input	Present State	Next State	Output
0	A	B	0
0	B	B	d
0	C	A	1
0	D	d	1
0	E	F	0
0	F	d	d
1	A	D	0

Input	Present State	Next State	Output
0	B	D	d
0	C	E	d
1	D	E	d
1	E	d	1
1	F	d	d

6.9 Simplify the following in-completely specified state transition table in Table 6.44 by the implication chart method. The in-completely specified state transition table contains 5 states: "A", "B", "C", "D" and "E". (a) Draw the implication chart. (b) Check the compatibility of the next states from the implication chart. (c) Draw the merger diagram of the compatible state pairs. (d) Check the coverage property of the compatible state classes. (e) Check the closure property of the compatible state classes. (f) Replace the compatible state classes with the new state names in the in-completely specified state transition table.

Table 6.44 In-completely Specified State Transition Table of Exercise 6.9

Input	Present State	Next State	Output
0	A	D	d
0	B	B	0
0	C	D	0
0	D	C	d
0	E	C	1
1	A	A	d
1	B	A	d
1	C	B	d
1	D	C	d
1	E	B	d

6.10 Design the counter for the 3-bit binary numbers by J-K flip-flop. The working mode of the counter is controlled by the input variable "C". If the input variable "C" is "0", the binary numbers are counted in the cycle "000→110→010→011→111→000". If the input variable "C" is "1", the binary numbers are counted in the cycle "000→100→110→111→011→000". (a) Write the state transition truth table. (b) Draw the K-map and the K-circles. (c) Write the state equation and the excitation equation. (d) Draw the logic circuit diagram.

6.11 Design the addition counter for 8 binary numbers by D flip-flop. The addition counter has an

- output variable "Z". If the addition counter generates the carry, the output variable "Z" is "1". Otherwise, the output variable "Z" is "0". (a) Draw the state transition diagram and write the state transition truth table. (b) Draw the K-map and the K-circles. (c) Write the state equation, the excitation equation and the output equation. (d) Draw the logic circuit diagram.
- 6.12 Design the subtraction counter for 6 binary numbers by J-K flip-flop. The subtraction counter has an output variable "Z". If the subtraction counter generates the borrow, the output variable "Z" is "1". Otherwise, the output variable "Z" is "0". (a) Draw the state transition diagram and write the state transition truth table. (b) Draw the K-map and the K-circles. (c) Write the state equation, the excitation equation, and the output equation. (d) Check whether the subtraction counter is self-starting. (e) Check whether the subtraction counter has the output error. (f) Draw the logic circuit diagram.
- 6.13 Use the integrated 4-bit addition counter with serial number "74LS161" to implement the addition counter for 7 binary numbers by the "clearing" method. Draw the wire connections in the chip diagram of the integrated counter.
- 6.14 Use the integrated 4-bit addition counter with serial number "74LS161" to implement the addition counter for 7 binary numbers by the "loading" method. Draw the wire connections in the chip diagram of the integrated counter.
- 6.15 Use two chips of the integrated 4-bit addition counter with serial number "74LS161" to implement the addition counter for 160 binary numbers. Draw the wire connections in the chip diagram of the integrated counter.

Design the addition counter for 8 binary numbers by J-K flip-flop. The subtraction counter has an output variable "Z". If the subtraction counter generates the borrow, the output variable "Z" is "1". Otherwise, the output variable "Z" is "0". (a) Draw the state transition diagram and write the state transition truth table. (b) Draw the K-map and the K-circles. (c) Write the state equation, the excitation equation, and the output equation. (d) Check whether the subtraction counter is self-starting. (e) Check whether the subtraction counter has the output error. (f) Draw the logic circuit diagram.

Chapter Seven

Verilog Implementation of Logic Circuit

In this chapter, we will introduce the basic programming grammar of the hardware description language "Verilog". Also, we will introduce the Verilog implementation of the logic gates, the Verilog implementation of the combinational logic circuit, the Verilog implementation of the flip-flop, and the Verilog implementation of the synchronous sequential logic circuit.

7.1 Basic Programming Grammar of Verilog

In this section, we will introduce the basic programming grammars of Verilog, including the data type, the identifier, the two basic variable types, the operator, and the three basic description methods.

7.1.1 Data Type

The Verilog definition of the decimal, binary, octal, or hexadecimal integer number is illustrated in Table 7.1. The "bit width" is the number of binary bits occupied by the defined integer number. If the bit width is omitted, the integer is defined by its default number of bits. The lower case letter "s" or the upper case letter "S" indicates that the integer is a signed number. If letter "s" or "S" is omitted, the integer is defined as an unsigned number by default. The letter "d", "b", "o" or "h" indicates that the integer is a decimal, binary, octal, or hexadecimal number. The value of the defined integer must be given. The definitions inside the square brackets "[]" can be omitted, but the definitions inside the angle brackets "< >" must be specified.

Table 7.1 Definition of the Decimal, Binary, Octal, or Hexadecimal Integer Number

[Bit Width]	'	["s" or "S"]	< "d", "b", "o" or "h" >	< Value of Integer >
---------------	---	----------------	--------------------------	----------------------

Some examples of the Verilog definition of decimal, binary, octal, and hexadecimal integers are illustrated in Table 7.2.

Table 7.2 Examples of the Verilog Definition of Decimal, Binary, Octal, and Hexadecimal Integers

Integer	Definition
4'd4	Define a 4-bit unsigned decimal integer "4" (its equivalent binary number is "0100")

“CP” and “Reset” are inside the parentheses “()”. The sequence detector is triggered by the falling-edge “CP”. So the sensitive variable “CP” has the key word “negedge”. The input “Reset” is triggered by the low-level voltage. So the sensitive variable “Reset” has the key word “negedge”. The sensitive variables “CP” and “Reset” are separated by the key word “or”. The “if-else” statement inside the 1st “always” statement is executed repeatedly.

(b) The “if-else” statement has 2 branches.

In the 1st branch, if the input “Reset” is “0”, the present state “State_Present” is reset to the initial state “A” by the nonblocking assignment.

In the 2nd branch, the present state “State_Present” transfers to the next state “State_Next” by the nonblocking assignment.

(2) The 2nd “always” Statement

(a) The 2nd “always” statement has the key word “always @”, and its sensitive variables “State_Present” and “IN” are inside the parentheses “()”. The sensitive variables “State_Present” and “IN” are separated by the key word “or”. The “case” statement inside the 2nd “always” statement is executed repeatedly. The “case” statement begins with the key word “case” and ends with the key word “endcase”.

(b) The present state “State_Present” is the condition of the “case” statement. The “case” statement has 4 branches. In the first 3 branches of the “case” statement, we write the “if-else” statement as the sub branch.

In the 1st branch of the “case” statement, the present state “State_Present” is the initial state “A”. If the input “IN” is “0”, the next state “State_Next” is still “A”. If the input “IN” is “1”, the next state “State_Next” transfers to the state “B”.

In the 2nd branch of the “case” statement, the present state “State_Present” is the state “B”. If the input “IN” is “0”, the next state “State_Next” transfers to the initial state “A”. If the input “IN” is “1”, the next state “State_Next” transfers to the state “C”.

In the 3rd branch of the “case” statement, the present state “State_Present” is the state “C”. If the input “IN” is “0”, the next state “State_Next” transfers to the initial state “A”. If the input “IN” is “1”, the next state “State_Next” transfers to the state “C”.

In the 4th branch of the “case” statement, the present state “State_Present” is none of the defined states “A”, “B” or “C”. The next state “State_Next” transfers to the initial state “A” by default.

Exercise

7.1 Implement the full adder of two 1-bit binary numbers in Verilog.

- (1) Use the dataflow level description to implement the logic functions of both the sum and the carry for the higher-bit.
- (2) Use the gate level description to implement the logic functions of both the sum and the

carry for the higher-bit.

7.2 Implement the voting circuit of 7 people in Verilog.

(1) If more than 3 people agree, the decision is approved. Otherwise, the decision is rejected.

(2) Use the behavior level description to implement the logic function of the voting circuit.

7.3 Implement the T flip-flop with the input variables "Set" and "Reset" in Verilog.

(1) The T flip-flop is triggered by the falling-edge clock pulse.

(2) The input variables "Set" and "Reset" are triggered by the low-level voltages.

(3) Use the behavior level description to implement the logic function of the T flip-flop.

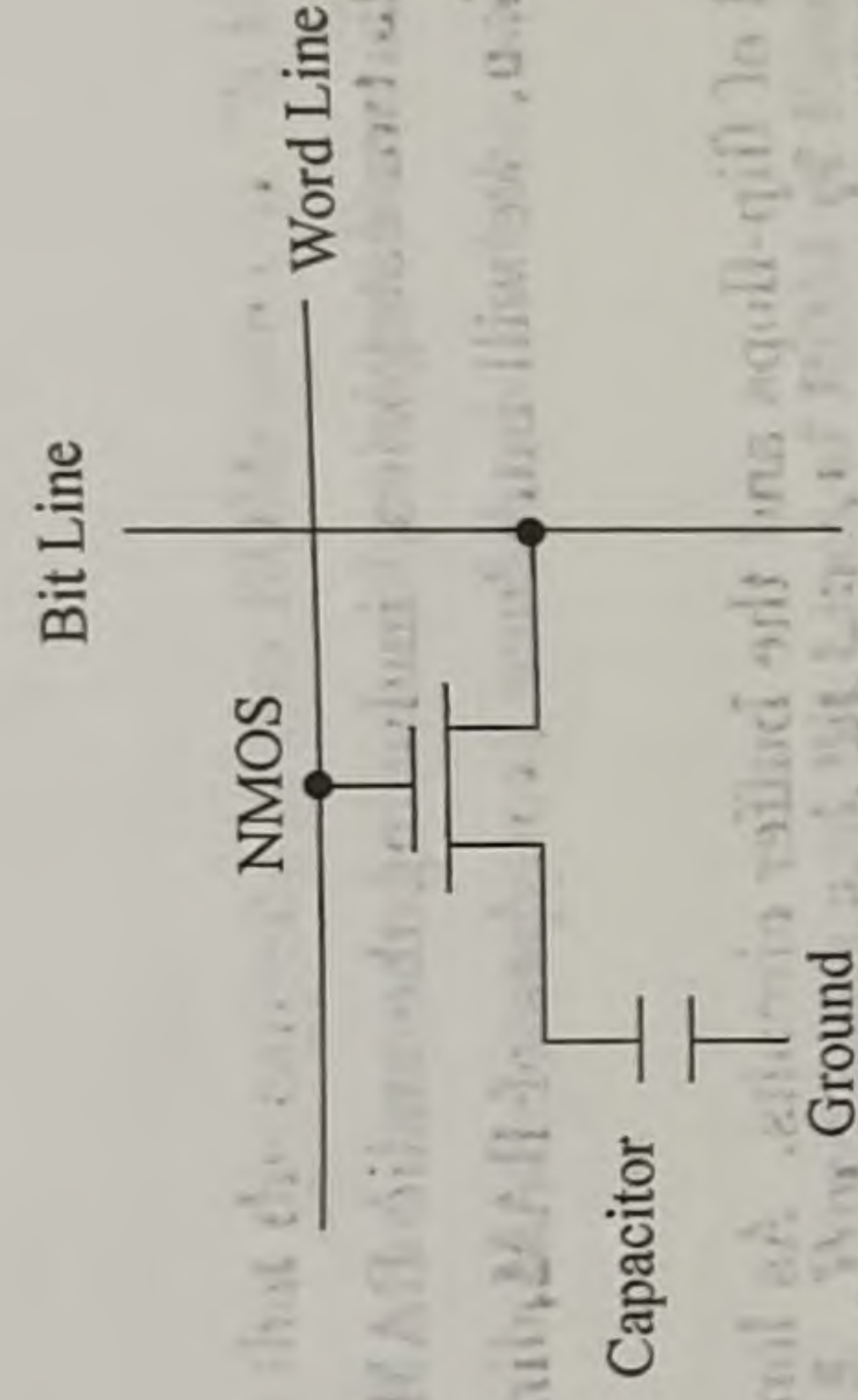


Figure 8.6 Example of DRAM

Exercise

8.1 Implement the following logic functions by the “AND-OR array” of ROM, and calculate the capacity of ROM.

$$Y_1 = \overline{A} \cdot \overline{BC} + A \overline{B} \cdot \overline{C} + A \overline{BC} + A \overline{B} C$$

$$Y_2 = \overline{A}BC + A\overline{B}C + ABC$$

8.2 Implement the following logic functions by the “AND-OR array” of ROM, and calculate the capacity of ROM.

$$F_1 = \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot \bar{B} \cdot C + A \bar{B} \bar{C} + A B C$$

$$F_2 = \overline{A} \cdot \overline{BC} + A \overline{BC} + \overline{AB} C + \overline{ABC}$$

$$F_3 = AC + B$$

018/01